NBA Bounce — Team Identity Texture Guide

Reference for the Mod Manager: which textures drive a team's jersey, logos, and court

How Team Identity Is Built

A team's visual identity in NBA Bounce is spread across roughly 17-21 separate textures per team, stored mainly in resources.assets and sharedassets1.assets, with a few UI-only pieces in sharedassets2.assets. Some textures are duplicated in two files (notably the base jersey shirts) — both copies must be replaced or the jersey will look correct in one context (e.g. exhibition) and revert in another (e.g. Association mode).

Team names in the catalog use camelCase for avatar/jersey textures (e.g. losAngelesLakers) and PascalCase with no spaces for court/arena textures (e.g. LosAngelesLakers). Historic/relocated franchises — New Jersey Nets, Vancouver Grizzlies, Charlotte Bobcats, Seattle Sonics — and several fictional courts (Jungle, Ice, Magma, Wind, GOAT, East/West Special, Nintendo) also follow this same texture structure.

Texture Categories

Identity Component	Texture Naming Pattern	Asset File(s)	Size	What It Controls
Court Floor (base)	txt_bounce_court_{TeamName}_D	resources.assets	2048×256	Center-court wordmark/logo decal overlaid on the floor — mostly transparent. Does NOT control the floor's base color or wood-grain pattern; see "Court Colors" and "Court Floor Patterns" below.
Court Floor (retro/alt logo)	txt_bounce_court_{TeamName}_D_retroLogo_# _team_##	resources.assets	2048×256	Alternate center-court logo eras for throwback/classic court variants.
Arena / Tunnel Signage	txt_bounce_publi_{TeamName}_D (+ retroLogo variants)	resources.assets	2048×256	Exterior and tunnel-area branding shown around the arena, not the floor itself.
Player Jersey (base)	txt_avatar_{teamName}_001	resources.assets, sharedassets1.assets	1024×1024	Default jersey skin applied to the player 3D model. Exists in two files — both need updating.
Player Jersey (color variants)	txt_avatar_{teamName}_shirt01 / _shirt02 / _shirt03	resources.assets, sharedassets1.assets	1024×1024	Home / away / alternate jersey color sets.
Player Jersey (NBA edition)	txt_avatar_{teamName}Association_001 / {teamName}Statement_001	resources.assets	1024×1024	Licensed NBA jersey editions (Association, Statement).
Classic / Throwback Logo	txt_classicLogos_{team}00N	sharedassets1.assets	1024×1024	Retro logo art used for classic-team unlocks and historic uniforms.
Team Menu Logo (large)	{TeamAbbr}_Global	sharedassets1.assets	1024×1024	Primary team logo shown in menus and team-select. Also doubles as the court's center-logo mask — see `_Texture_Logo_Mask` below.
Team Menu Logo (small)	{TeamAbbr}_Global_Icon	sharedassets1.assets	512×512	Compact logo used in rosters, HUD labels, and lists.
Unlockable Content Icon	{TeamAbbr}_Unlock_01 / _02 / _03	sharedassets1.assets	1024×1024	Icons representing unlockable alternate jerseys/courts for that team.
Mascot	txt_mascot_{teamName}	sharedassets1.assets	2048×2048	Team mascot artwork.
Team Basketball Skin	txt_bball_diffuse_{teamName}_001	sharedassets1.assets	512×512	Team-colored basketball skin used in-game.

Identity Component	Texture Naming Pattern	Asset File(s)	Size	What It Controls
Team Selector Thumbnail	e.g. angeles_lakers	sharedassets2.assets	400×400	Small thumbnail shown on the team-select screen.
Team Customization Banners	txt_ui_teamSelector_upperBanner, txt_ui_editTeam_upperBannerNew, txt_ui_editTeam_lowerBanner	sharedassets2.assets	varies	Backdrop banners in team-select/edit-team UI — shared assets, not unique per team.
In-Game Scoreboard	txt_ui_ingame_scoreboard, _scoreboardGoldHome, _scoreboardGoldVisitor	sharedassets2.assets	1036×157	HUD scoreboard background — generic styling, not team-specific.

Worked Example: Los Angeles Lakers

Every texture below carries Lakers branding (purple/gold, logo, mascot). To fully reskin the Lakers — as a new team, a fictional team, or a different color scheme — every row in this table is a candidate for replacement.

Texture Name	Asset File	Size	Role
txt_avatar_losAngelesLakers_001	resources.assets	1024×1024	Base jersey
txt_avatar_losAngelesLakers_shirt01	resources.assets	1024×1024	Jersey color variant
txt_avatar_losAngelesLakers_shirt02	resources.assets	1024×1024	Jersey color variant
txt_avatar_losAngelesLakers_shirt03	resources.assets	1024×1024	Jersey color variant
txt_avatar_losAngelesLakersAssociation_001	resources.assets	1024×1024	NBA Association edition jersey
txt_avatar_losAngelesLakers_shirt01 / 02 / 03	sharedassets1.assets	1024×1024	Duplicate jersey refs — must match resources.assets
txt_bounce_court_LosAngelesLakers_D	resources.assets	2048×256	Court floor decal (base)
txt_bounce_court_LosAngelesLakers_D_retroLogo_4_team_23	resources.assets	2048×256	Court floor decal (retro logo variant)
txt_bounce_publi_LosAngelesLakers_D	resources.assets	2048×256	Arena/tunnel signage
txt_bounce_publi_LosAngelesLakers_D_retroLogo_4_team_23	resources.assets	2048×256	Arena/tunnel signage (retro variant)
txt_classicLogos_losangelesLakers001	sharedassets1.assets	1024×1024	Classic/throwback logo
LALakers_Global	sharedassets1.assets	1024×1024	Team menu logo (large)
LALakers_Global_Icon	sharedassets1.assets	512×512	Team menu logo (small)
LALakers_Unlock_01 / 02 / 03	sharedassets1.assets	1024×1024	Unlockable content icons
txt_mascot_losAngelesLakers	sharedassets1.assets	2048×2048	Mascot artwork
txt_bball_diffuse_losAngelesLakers_001	sharedassets1.assets	512×512	Team basketball skin
angeles_lakers	sharedassets2.assets	400×400	Team selector thumbnail

Minimum Reskin Checklist

To change one team's identity end-to-end, replace at minimum:

1. Base jersey + shirt01/02/03 (in both resources.assets and sharedassets1.assets)
2. Association/Statement edition jerseys, if the team has them
3. Court floor decal (base) and any retroLogo court variants
4. Arena/tunnel signage (publi) base and retroLogo variants

5. Classic/throwback logo
6. Global logo (large) and Global_Icon (small)
7. Unlock_01/02/03 icons
8. Mascot artwork
9. Team basketball skin (bball_diffuse)
10. Team selector thumbnail
11. Court floor colors and floor pattern — see the two sections below

Scoreboard, crowd, hoop/rim, and arena-shell textures are shared across all teams and don't need per-team changes unless you're restyling the whole game rather than one team.

Court Colors — Material Properties, Not Textures

While building the Mod Manager's Court Colors feature, direct inspection of the game files found that the court floor's actual color and the out-of-bounds/apron color are NOT controlled by any texture listed above. txt_bounce_court_{Team}_D turns out to be a mostly-transparent decal carrying only the center-court wordmark and logo artwork; the wood-grain floor texture underneath it (txt_bounce_parquet01_D, 512×512) is shared identically across every team *by assignment* — though, as the next section shows, that assignment is per-material and can be changed.

Per-team floor and apron color live in named Color and Float properties on two Unity Material objects per team — mat_bounce_stadium_parquet_{Team} and mat_bounce_stadium_lines_{Team}, both in sharedassets1.assets — using the shared shader Shader Graphs/shd_bounce_courtLit_v2. These can be patched in place the same way texture pixels are, since overwriting an existing fixed-size Color/Float field doesn't change the object's byte size.

Confirmed Properties

Property	Material	Status	Notes
_Color_Parquet	mat_bounce_stadium_parquet_{Team}	**Working**	Floor Color. Tints the wood-grain texture and preserves the grain, but only when _Court_Lines_ON_OFF = 0 on this material.
_Color_Area_Line_R	mat_bounce_stadium_parquet_{Team}	**Working**	Out-of-Bounds Color (apron beyond the sideline). Only renders when _Court_Lines_ON_OFF = 1 on this material.
_Court_Lines_ON_OFF	mat_bounce_stadium_parquet_{Team} (float)	**Master switch**	0 = floor-tint mode, wood grain intact. 1 = out-of-bounds mode, but the floor falls back to its default look. The two properties above can't both show custom colors at once.
_Color_Outside_Court_R	mat_bounce_stadium_parquet_{Team}	**Non-functional**	Gated behind a disabled, unused texture slot (_Outside_Texture_ON_OFF = 0). No visible effect regardless of value.
_Color_Restricted_Line_A	parquet and lines materials	**Non-functional**	Doesn't drive the restricted-arc line under the basket on either material, in any combination tested (including the exact recipe that works for Out-of-Bounds Color). The arc's color is very likely set by game code at runtime, not baked into the Material — not reachable through asset patching alone.

Alpha is not simple transparency on this shader — it behaves as part of a blend/mode switch tied to _Court_Lines_ON_OFF, not a 0–1 opacity slider.

In the Mod Manager

This is exposed directly in the app via the ■ Court Colors button — no manual asset editing required. It includes a custom color picker with a Photoshop-style eyedropper, and automatically manages the _Court_Lines_ON_OFF trade-off: it only forces lines-on when

Out-of-Bounds Color is actually changed, so editing Floor Color alone keeps the wood grain intact.

Sprite Crops — why a replacement logo comes out cut off

Verified against sharedassets1/2 and resources.assets. This is the mechanism behind the sprite un-crop that now runs automatically on Apply.

Replacing a logo texture is only half the object graph. Every logo in the game is drawn through a Unity **Sprite** that sits in front of the Texture2D, and all of them were imported with **Mesh Type = Tight**. Two things were therefore baked at build time from the *original* artwork's opaque pixels:

Baked field	What it holds	Example — GSWarriors_Global
`m_RD.textureRect`	Tight crop box around the old logo, in pixels, bottom-left origin	514×619 at (259, 193) inside a 1024×1024 texture
`m_RD.m_VertexData` / `m_IndexBuffer`	Polygon hull of the old logo's silhouette	250 vertices / 879 indices
`m_RD.settingsRaw` bit 6 (=64)	meshType: 1 = Tight, 0 = FullRect	64

UGUI maps the sprite's `m_Rect` (the whole canvas) onto the on-screen box and pads the difference — `DataUtility.GetPadding` — so **only the pixels inside `textureRect` are ever sampled**. Replacing the texture pixels leaves that box describing the logo that used to be there. A replacement covering more of the canvas than the logo it replaced — exactly what happens when a wide wordmark replaces a round primary logo — is silently clipped to the old logo's bounding box, even though it lined up perfectly in Photoshop on top of the exported original.

This is not a rare case. Sprites front nearly every identity texture, and almost all of them are trimmed:

File	Sprites	Trimmed crop	Tight mesh	Multi-sprite atlases
sharedassets1.assets	350	341	349	1
sharedassets2.assets	326	255	325	1
resources.assets	62	57	57	0

Jersey textures (`txt_avatar_*`), court decals and arena signage have **no** sprite in front of them, so they were never affected — the problem is specific to the menu logos, icons, unlock art and classic logos.

The patch

Point the crop at the whole texture and drop the baked hull:

- `textureRect` → `(0, 0, m_Width, m_Height)`, `textureRectOffset` → `(0, 0)`
- `settingsRaw` → clear bit 6, so meshType becomes FullRect
- rewrite the tight hull as one full-rect quad: first four vertices become the rect's corners with UV (0,0)–(1,1), every later vertex is a copy of the first, the first six indices draw the quad and every remaining index is zeroed into a degenerate, never-rasterised triangle

Vertex streams are laid out in stream order, each padded up to a 16-byte boundary (`m_VertexData.m_DataSize` = 5008 bytes for 250 verts: 250×12 → 3008, then 250×8). Refilling both buffers at their existing length keeps them byte-identical in size, and everything else overwritten is a fixed-width float or int — so the reserialized Sprite comes out the same byte size and splices into the `.assets` file in place, exactly like the texture and material patches above.

Multi-sprite textures are left alone. Where several sprites share one texture (`txt_assets_decorationAnimationGOAT_D` carries five), each `textureRect` is a real sub-region and widening one would move the others' artwork. The Mod Manager detects this on import and lists the regions the replacement has to stay inside instead.

In the Mod Manager

`sprite_crop.py`, self-contained — it imports nothing from app.py. Selecting a texture outlines its crop box on the **Original** preview with a dashed rectangle and states it in the info line, so it's visible before authoring anything that the old logo only used the middle 514×619 of its canvas. Apply then widens the crop automatically and reports how many sprites it touched; Remove Mod puts the texture and its sprite back to stock byte-for-byte from the backup without disturbing other mods.

Related: streamed pixel data and multiple mods in one file

Worth recording alongside the above, since it produces the same *symptom*. Every Texture2D in this game is streamed — 1,140 of 1,140 in sharedassets1.assets, 431 of 434 in resources.assets carry an `m_StreamData.path`, with pixels living in the companion `.resS`. `apply_single_mod()` appends the new pixels to the `.resS` and repoints `offset`/`size`, which means the `.resS` may only be rewound to its backup **once per Apply run**. Rewinding it per mod — as the app used to — discards the bytes of every mod already applied to the same file and leaves those textures pointing at a stale offset, often past the new end of file. Reproduced with two 1024×1024 mods in sharedassets1.assets: both objects ended up at offset 122,221,152 and both rendered the second mod's image. Since a full team reskin touches ~17 textures across two files, this hit essentially every multi-texture mod.

Court Floor Patterns — the `_Texture_Parquet` Slot

Verified in-game. This is the mechanism behind the Floor Patterns feature.

The wood-grain floor texture is not hardcoded into the shader. Every stadium material carries its own `_Texture_Parquet` texture slot, and the game already ships four courts using it to show completely different floors.

What the probe found

All 80 stadium materials (both the `parquet` and `lines` material for each of ~38 courts, plus the GOAT courts) have a populated, non-null `_Texture_Parquet` PPtr. Five distinct textures are in use out of the box:

Texture	File	Size	Format	Used by
txt_bounce_parquet01_D	sharedassets1.assets (pathID 1760)	512×512	DXT1Crunched	**76 materials** — every NBA and classic court
txt_goat_iceCourt_smoothCracks	sharedassets2.assets (170)	2048×2048	DXT5	mat_bounce_stadium_parquet_IceCourt
txt_goat_jungleCourt_redClay	sharedassets2.assets (324)	1024×1024	DXT1Crunched	..._JungleCourt
txt_goat_magmaCourt_stoneTile01	sharedassets2.assets (365)	1024×1024	DXT1Crunched	..._MagmaCourt
txt_goat_windCourt_graniteTile	sharedassets2.assets (280)	1024×1024	DXT1Crunched	..._WindCourt

Per-court floor patterns are therefore a **supported, shipping mechanism**, not a hack. The 38 NBA courts simply all happen to point at the same texture.

The patch

The `m_TexEnvs` entry serializes as:

```
[int32 strlen][ "_Texture_Parquet" ][align to 4]
[int32 m_FileID][int64 m_PathID]     <- 12 bytes, the redirect target
[float2 m_Scale][float2 m_Offset]    <- 16 bytes
```

Locating it is deterministic: find `b"_Texture_Parquet"` inside the object's `byte_start .. byte_start + byte_size` slice, advance to the next 4-byte boundary, and the PPtr follows. This was verified against all 72 parquet/lines materials in sharedassets1.assets — every one decoded to `fileID 0, pathID 1760`, with zero failures.

Writing 12 bytes is size-neutral, so it patches in place exactly like the Color/Float work above. `apply_single_mod()` is not involved.

Reachability — which files a court can point at

A PPtr's `fileID` can only name a file already in that material's externals list:

Material lives in	Can reference
sharedassets1.assets (all NBA/classic courts)	itself (0), globalgamemanagers (1), unity_builtin_extra (2), unity default resources (3), **resources.assets (4)**
sharedassets2.assets (GOAT/special courts)	the above, **plus sharedassets1.assets (5)**

sharedassets2 is not reachable from the NBA court materials. That is why the GOAT courts, not the NBA ones, are the courts pointing at sharedassets2 textures today.

Donor slots — the eight-pattern ceiling

New Texture2D objects cannot be created without changing file layout, so custom patterns must reuse existing texture objects. A full reference scan across all 14 serialized files (2,023 Texture2D objects, ~290k objects walked) found **zero** unreferenced textures in sharedassets1.assets, but eight perfect candidates in resources.assets: documentation screenshots left behind by a localization asset-store plugin.

Donor	pathID	Slot bytes
add_new_view	288	174,776
api_quick_start	402	174,776
langScreenshotColId	100	174,776
lang_deleteLang	339	174,776
lang_selectLang	309	174,776
setup_setting_mw	392	174,776
st_build_settings	131	174,776
str_editor	261	174,776

All are 512×512 plain DXT1. 174,776 bytes is exactly a complete 512×512 DXT1 mip chain (131072 + 32768 + 8192 + 2048 + 512 + 128 + 32 + 8 + 8 + 8), so a pattern authored as 512×512 DXT1 with 10 mips drops in with **zero padding and zero size delta** — cleaner than editing the stock texture, which is *crunched* and therefore variable-length.

Ten more dead plugin screenshots exist at smaller sizes (512×256 down to 512×32) if lower-resolution slots are ever useful.

This gives 8 simultaneous custom patterns plus the untouched stock floor. The pattern *library* is unlimited; eight is how many can be live across all courts at once.

If more than eight are ever needed

`m_Width`, `m_Height`, `m_TextureFormat`, `m_MipCount`, `m_CompleteImageSize` and `m_StreamData.offset/size` are all fixed-width integer fields, patchable in place. Any texture with a slot ≥ 174,776 bytes can therefore be reshaped into a 512×512 DXT1 donor — the `txt_bounce_court_*_retroLogo_*` decals are 524,288 bytes each and there are dozens, at the cost of sacrificing retro court logos.

Further out: `m_StreamData.offset` points into the `.resS`, which is a separate blob. Appending to the end of a `.resS` and repointing offset/size shifts nothing in the `.assets` file — untested, but if it holds, texture size stops being a constraint entirely.

Shader properties that affect the floor pattern

Read from `mat_bounce_stadium_parquet_BostonCeltics`:

Property	Type	Value	Effect
_Texture_Parquet	TexEnv	pathID 1760	The floor pattern texture itself.
_Tiling_Parquet	Color	(0.400, 0.405, 0, 0)	**Tiling density**, r and g used as a UV scale pair. Patchable in place like any other color — tiling is a slider, not a re-export.
_Rotation_Parquet / _Rotate_Parquet	Float	0.0	Pattern rotation.
_Offset_Parquet	Color	(0.320, 0, 0, 0)	Pattern UV offset.
_Contrast_Parquet	Color	(0, 1, 0, 0)	Pattern contrast response.
_Parquet_Strength_Normal	Float	0.05	The pattern also drives a subtle normal/relief response, so grooves pick up light.
_Parquet_Contrast_Normal	Color	(-3.08, 1, 0, 0)	Normal-map contrast.
_Parquet_Specular_Color / _Influence	Color	(0.509^3) / (0, 0.5, 0, 0)	Floor sheen.
_Parquet_Smoothness_Influence	Float	0.5	Gloss response.
_GOAT_Textures_ON_OFF	Float	0.0	1 on the GOAT courts. Likely switches sampling for the larger fictional-court textures.
_ONLY_PARQUET	Float	0.0	Isolates the parquet layer.
_Texture_Logo_Mask	TexEnv	{TeamAbbr}_Global	Per-team center-court logo mask — the team's menu logo doing double duty. Populated on 41 of 80 materials.

Confirmed in-game behaviour

Tested on the Celtics court with a deliberately garish validator texture, then with a real parquet:

- The pattern applies to the **in-bounds floor only**. The out-of-bounds apron is unaffected and stays on its own color.
- Court lines, the key, and the center-court logo decal all still render **on top** of the pattern.
- The texture is **multiplied by** `_Color_Parquet` — a white line in the source rendered tan at Boston's `(0.851, 0.621, 0.447)`. Patterns should therefore be authored **greyscale**, leaving the Court Colors picker to supply the wood tone. Pattern and colour stay independent.
- The stock texture tiles roughly **6× across the court**, so one tile covers about 16 feet. A 4×4 block grid per tile puts parquet blocks at roughly 4 feet — close to the real Boston Garden's 5-foot panels.
- Patterns must be **seamless**. The validator's border showed the tile grid clearly; a non-tiling image will show a visible lattice across the floor.

In the Mod Manager

Exposed via the **Floor Patterns** window (`floor_patterns.py`, self-contained — it imports nothing from app.py and never touches `apply_single_mod()`). It ships nine procedural, seamless, greyscale patterns based on real arena hardwood layouts, supports importing custom PNGs, previews each pattern tiled and tinted with the selected court's actual `_Color_Parquet`, allocates donor slots automatically, and warns when the eight-slot ceiling is reached.

Stock floor as a template. On first run the window decodes `txt_bounce_parquet01_D` out of the game — from `sharedassets1.assets.original_backup` when one exists, so a previously applied pattern can't be mistaken for the original — and caches it as a PNG. The cache is never shipped with the app: it is game art, so it has to come from the user's own install. Extraction (`extract_stock_pattern()`) needs nothing but a game path — it matches path ID 1760, falling back to the texture name — so the template survives a failed donor probe, and with no game folder configured the entry is simply absent rather than faked. It appears first in the library, so the shipped floor can be viewed and exported like any other pattern. It turns out to be **staggered planks in greyscale**, which independently confirms the greyscale-plus-tint design. Selecting it and assigning simply resets the court, and costs no slot.

Export for editing. Any pattern, the stock floor included, exports to a 512×512 PNG. An optional 2×2 sheet (1024×1024) writes four copies tiled together so seams at the tile edge are obvious while editing — crop back to a single 512 quadrant before re-importing. The

export dialog restates the three rules: 512×512, greyscale, seamless.

All writes are journaled with the pristine bytes recorded on first touch, so Revert restores the exact original state regardless of how many applies have stacked — and without disturbing any other mods.